Cursor implementation and verification playbook

COMET-XAI Lite • eight weeks • laptop/free Colab • approximately $20 Cursor plan

This is an operational guide for a beginner. It includes exact prompts, concrete files and acceptance criteria. Cursor-generated code is not evidence until you run it, inspect it and check it independently. Subscription limits and available model names change; this guide does not promise unlimited requests or a particular model entitlement.

1. What is already provided

The code/ folder contains a working preliminary UCI logistic baseline, nine checks, a data download script, frozen direct dependencies, a Colab notebook and saved results. The current research study is not complete. The baseline deliberately excludes neural networks, LIME, TreeSHAP and recourse.

Open the code/ folder in Cursor using File → Open Folder. You can copy its contents into your empty new-repo working checkout after reviewing them. Nothing has been pushed, committed or published by this task. Keep the original source PDF outside any public repository unless you have permission to publish it.

2. Run the existing demonstration first

Use Python 3.12, matching the verified cloud run. A different supported Python version may work but is not verified here. On Windows, install official Python and check “Add Python to PATH”. Open Cursor's Terminal → New Terminal. Commands are run from the folder containing demo.py.

Windows PowerShell:

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe download_data.py
.\.venv\Scripts\python.exe -m unittest -v test_demo.py
.\.venv\Scripts\python.exe demo.py --bootstraps 10 --random-partitions 30
```

macOS/Linux:

```
python3.12 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python download_data.py
.venv/bin/python -m unittest -v test_demo.py
OPENBLAS_NUM_THREADS=2 OMP_NUM_THREADS=2 .venv/bin/python demo.py --bootstraps 10 --random-
partitions 30
```

Using the virtual-environment Python path avoids activation-policy problems. Do not disable PowerShell security or TLS verification to make commands run. If Python 3.12 is not installed, use the Colab route for tomorrow, or install it from the official source. Installation of the pinned dependencies on your laptop is still to be verified; the cloud run used these versions already installed.

Read results/pilot.json. It must say “preliminary single-split real-data logistic baseline”. Confirm 30,000 rows, 24,000 train, 6,000 test, 500 audit, ten bootstrap fits and 30 random partitions. Approximate recorded values: AP .4894, ROC-AUC .7130, Brier .1477, group RBO .9240 and matched-random RBO .8631. Minor cross-platform floating-point differences can occur; large differences require investigation.

Do not type the terminal prompt symbols into the terminal. A command finishing with an error is not success. If data access fails, read the error and obtain the official UCI archive manually using the linked

source; do not replace it with fabricated data.

3. Colab route and offline backup

Upload `Colab_Quickstart.ipynb` to Google Colab, select a CPU runtime, and follow its upload instructions for the small code files. It downloads UCI data, installs pinned direct dependencies, runs tests and executes the pilot. Save results to your computer before the session ends. Colab notebook generation and underlying commands are verified locally; execution inside your personal Colab account has not been performed.

Free Colab sessions can disconnect. A GPU is unnecessary. Do not make tomorrow's presentation depend on network access or a freshly installed environment. Bring the four-slide PDF and saved results. If live execution fails, show the recorded run with its status and explain the failure honestly.

4. Use Cursor economically

Work one small module at a time. Give the relevant files and a bounded task rather than pasting the entire dossier into every conversation. Use a capable reasoning model for design/review and an available cheaper or automatic option for routine edits if your plan supports it. Inspect Cursor's current usage page for limits; do not assume the monthly fee covers all usage.

Ask for a short plan, then a small patch, then a terminal command and evidence. Reuse your project rules and accepted design instead of repeatedly regenerating the repository. Before asking for a rewrite, save a Git commit locally or copy a backup. Never let the agent delete results or overwrite raw data to resolve a failing test.

5. Permanent project instruction prompt

Paste this into a project instruction file supported by your Cursor version, or the opening message of each coding session:

```
We are building COMET-XAI Lite: a controlled audit of grouped feature-attribution
stability in tabular credit models. Read the research specification before editing.
I am a beginner. Explain the purpose of each changed file in plain language.
Scope: logistic baseline, one tree model, controlled synthetic data, UCI credit,
and optionally a second licensed credit dataset. No recourse or temporal claims.
Use the existing checkout. Do not create worktrees unless I request one.
Never fabricate data, results, references, passed tests, timings or screenshots.
Never tune on test data, fit preprocessing on test data, or silently drop failures.
Keep protected and identifier fields out of model inputs as specified in the data card.
Keep raw data immutable. Record source URL, licence, SHA256, split IDs and seeds.
Call summed feature attributions coalition sums, not Owen or exact group Shapley.
Specify model, output scale, masker, background and class for every attribution.
Use size-matched random partitions as controls. Stability alone is not correctness.
Keep numerical tests independent of the implementation; never weaken assertions
simply to make them pass. Do not silently replace libraries or disable verification.
Run commands when possible and quote real exit status. Otherwise label them unrun.
Do not push, publish, upload data, buy services or expose secrets without my request.
For each task give: files changed, exact run command, actual checks, remaining limits.
```

6. Phase A: understand and protect the baseline

Prompt A1 — explain existing code without editing it

```
Read demo.py, test_demo.py and results/pilot.json. Do not edit anything yet.
Explain the code in execution order, with one small numeric example for the
linear attribution formula. Show the exact lines that prevent train/test leakage,
fit the preprocessing pipeline, calculate correlations, create matched random
partitions, and verify logit additivity. Explain why the 45 pairs are dependent.
List what this pilot has not implemented. Then ask me five questions to check
whether I understand it. Wait for my answers before making changes.
```

Acceptance: you can explain the ten steps in the study guide and distinguish logit attribution from predicted probability. If Cursor says this code already runs TreeSHAP or recourse, correct it using the source.

Prompt A2 — independent read-only code audit

```
Act as a skeptical reviewer. Read the baseline without changing code.
Check: target/ID leakage, split overlap, raw-data provenance, imputation and scaling,
numeric status-code limitations, class ordering, fixed raw background, exact linear
logit attribution, group-size matching, ranking/tie handling and failure behaviour.
Compare formulas against hand calculations, not against functions from the same
implementation. Report issues by severity with file/line and a concrete example.
Do not invent an error if behaviour is correct. Separate scientific limitations
from implementation bugs. Recommend the smallest patch only for confirmed bugs.
```

Acceptance: audit explicitly recognizes that numeric repayment-status coding is a baseline limitation; global stability is not local fidelity; a random-partition standard deviation is not a confidence interval.

Prompt A3 — make a clean reproducible project

```
Refactor only after preserving the current baseline results. Propose a small src/
layout for data.py, splits.py, models.py, attribution.py, grouping.py, metrics.py,
controls.py and experiments.py; tests/; configs/; artifacts/. Preserve all numerical
behaviour of the existing pilot. Add a CLI with --config and --output-dir. Refuse
to overwrite an existing run unless explicitly requested. Store config, seed,
package versions, data SHA256, split IDs, source commit if available and runtime.
Keep raw data outside Git; add a data README with licence and source. Run the old
baseline and refactored version and compare deterministic metrics. Do not change
the experiment or add new models in this task.
```

Acceptance: refactor reproduces old metrics; run directories are immutable; `git diff` contains no raw data or credentials. Do not spend week 1 building a complicated web application.

7. Phase B: trustworthy data and synthetic checks

Prompt B1 — data card and leak-proof splits

```
Implement the final 60/20/20 stratified train/development/test split, with a fixed
seed in configuration and immutable saved row IDs. Fit transforms on training
only. Document UCI target direction, unusual repayment-status values, omitted
fields, data licence and known limitations in data_card.md. Propose a categorical
encoding sensitivity for repayment status without claiming negative codes mean
the same thing. Keep all one-hot levels of one original variable together for
attribution grouping. Add tests that fail on overlapping split IDs, target leakage,
test-fitted scaling and unexpected schema. Keep the old pilot as a separate run.
```

Acceptance: intentionally insert a test row into train IDs and confirm a failure; changing only test values must not alter training preprocessing or clusters. Restore the correct files after testing the failure.

Prompt B2 — synthetic generators and a real oracle

```
Implement deterministic synthetic cases: independent additive score, correlated
proxies, redundant irrelevant inputs, known interaction, and opposing contributions.
Separate the data-generating process from the model being explained. Ground truth
must be the exact explanation of a stated finite-background game, not an assumed
coefficient ranking. Implement exhaustive Shapley over subsets for d<=8 and exact
group Shapley for M<=8 using factorial subset weights and cached value evaluations.
Reject larger exact requests with a helpful error. Test two-player values 20/30,
linear contributions 3/6, dummy-player zero, efficiency, symmetry, and the three-
player unanimity example: ordinary {A,B} sum=2/3, merged-group value=1/2.
Do not use the production attribution function to generate expected test answers.
```

Acceptance: all hand-derived games match within tolerance; interaction case demonstrates coalition sums differ from group values. A test suite with zero tests is a failure, even if its exit code is zero.

Prompt B3 — stress-test grouping

```
Implement training-only complete-linkage grouping at rho=.5,.7,.9. Handle constants,
NaNs, one-column input, invalid rho and grouped categorical levels explicitly.
Add average-linkage only as a separate valid alternative. Never run Ward on
1-abs(Spearman) distances. Test the within-group complete-linkage condition, total
feature coverage exactly once and the direction of group-count changes when rho
increases. Add a separate partition-stability analysis using co-membership matrices
after reclustering bootstrap samples. Do not confuse that with the fixed-partition
attribution experiment.
```

8. Phase C: models, explanations and fair controls

Prompt C1 — one tree model, no uncontrolled expansion

```
Add one small tree ensemble and interventional TreeSHAP. Before writing code,
inspect the installed SHAP/model API and propose the supported output scale and
background construction. Use a training-only background and the same held-out
audit IDs for all methods. Handle binary class dimensions explicitly by checking
classes_ and output shape. Add additivity checks against the exact explained output
with documented tolerance. If the library is incompatible, report the actual error
and propose a supported version combination; do not silently switch to a different
explanation target. Keep sample sizes small for CPU. Record library versions and
real runtime. No LIME or EBM in this task.
```

Acceptance: explanations reconstruct the model output; target class and scale are printed in metadata; no new test-data dependency. A successfully imported library is not proof explanations were computed correctly.

Prompt C2 — grouped summaries and matched controls

```
Given saved attribution tensors [model, applicant, feature], implement net group
contributions, mean absolute net importance, unsigned within-group mass and
cancellation ratio with undefined-zero handling. Preserve individual contributions
for drill-down. Generate at least 100 random partitions per final setting preserving
the exact multiset of group sizes. Use the same attribution tensors and model pairs
for every partition. Add no-grouping, giant-group and documented domain-group
controls. Compute RBO p=.85 and sensitivity .7/.95 with explicit tie handling.
Never directly interpret feature-versus-group RBO as a controlled effect. Save all
partition seeds and scores, not only their mean. Tests must verify every partition
uses each original feature exactly once and matches the size multiset.
```

Acceptance: giant-group control has trivial stability and is clearly excluded from claims of useful resolution; matched random controls use actual attribution values, not merely random ranking lists.

Prompt C3 — faithfulness and cancellation, including failures

```
Implement exact-group attribution error on the synthetic oracle cases, cancellation
diagnostics and a documented group-deletion response on held-out examples. Specify
the masker and assess implausible combinations; do not call marginal deletion
causal. Do not use a deletion metric both to tune weights and claim independent
validation. Make a plot showing stability versus fidelity/cancellation, including
bad synthetic cases. Keep high-cancellation groups visible with component drill-down.
If the grouping improves stability but worsens information quality, report both.
Do not delete unfavourable runs.
```

Prompt C4 — optional LIME, only after core gates

```
Add LIME only as a secondary experiment on the same fitted model and rows.
Inspect its actual standardized/discretized representation and class index. Explain
units and reconstruction residual. Set seeds, repeat sampling, record neighbourhood
and kernel width, and separate sampling variance from model retraining variance.
Do not multiply a standardized coefficient by an unscaled raw displacement.
Do not claim LIME and SHAP answer identical questions. Make implementation and
semantic differences explicit in metadata and the report.
```

9. Phase D: experiments, statistics and paper outputs

Prompt D1 — freeze an experiment plan

```
Produce an experiment manifest before running the final test set. Include datasets,
splits, model family and settings, bootstrap seeds, audit/background IDs, rho choices,
random-partition seeds, RBO persistence, fidelity metrics and primary comparisons.
Use development data only for selection. Explain the sampling unit for every
interval. Keep main comparisons small. Estimate runtime from a measured small run
and set CPU/thread/memory limits; do not launch the full Cartesian grid blindly.
Write a pre_run_checklist.md and stop for my review before final expensive runs.
```

This is your own project-management checkpoint, not a requirement to ask permission for every routine code edit. Review once at the experiment freeze, then run the accepted plan.

Prompt D2 — uncertainty without pseudoreplication

```
Implement paired method differences within each split and report all outer-split
summaries. Do not treat bootstrap model pairs or overlapping split scores as
independent datasets. If bootstrapping applicants with models fixed, label intervals
conditional-on-models. If bootstrapping models, recompute all pairwise statistics
inside each draw. Random-partition spread is not a confidence interval. Avoid a
Friedman significance headline on two datasets. Explain assumptions for any formal
test and apply Holm only to predeclared valid comparisons. Include sample counts,
effect sizes, skipped cases and reasons in every output table.
```

Prompt D3 — generate figures from evidence only

```
Create plots and paper tables only from immutable run JSON/CSV files. Every figure
must record source run IDs and configuration. Label preliminary, synthetic,
illustrative and measured values distinctly. Missing runs must say 'not run'; never
fill them with expected values. Produce a correlation heatmap, matched-control
stability comparison, cancellation/drill-down figure, synthetic failure example,
stability-fidelity frontier and runtime table. Export PDF/PNG with readable fonts.
Do not use significance stars without a valid saved statistical analysis.
```

Prompt D4 — reference and novelty audit

```
Read the closest full papers, especially groupShapley and the 2025 ITNAC feature-
grouping robustness paper. Build a claim-to-source matrix: exact claim, cited page
or section, evidence level, and how our work differs. Resolve current metadata
from publisher records; do not invent DOIs or access dates. If our contribution is
already covered, explicitly say so and propose a faithful replication or a genuinely
different evaluation question. Do not write 'first' or 'state of the art' without
supporting evidence. Draft only results backed by completed runs.
```

Prompt D5 — adversarial final review

```
Review this repository as if rejecting a paper for invalid evidence. Read-only first.
Look for data leakage, target leakage, incompatible attribution scales, incorrect
class selection, arbitrary ground truth, shorter-list artefacts, group-size mismatch,
ignored cancellation, hidden failures, pseudoreplication, test-set tuning, unsupported
causality/recourse/novelty and fabricated references. Verify table cells against raw
run artifacts. List confirmed defects separately from unresolved risks. Then suggest
minimal fixes and the exact checks needed to clear each defect.
```

10. How you personally verify generated code

Do not accept “all tests pass” in chat without terminal output. Run the exact command yourself and check its exit code. In PowerShell, `$LASTEXITCODE` reports the preceding native process status. In bash, `echo $?` immediately after the command does so. A nonzero value needs diagnosis.

For a scientific module, verify three independent things: a hand-calculated small example, a structural property that must hold, and a deliberate bad input that must be rejected. Example: the linear formula gives 3 and 6; contributions sum to the score difference; a wrong feature order triggers a schema error.

Inspect shapes and units: X is rows×features; probability is rows×classes; local attribution is rows×features after explicit class selection; grouping is a partition of feature IDs. Read warnings, especially failed convergence. A warning is not fixed by suppressing it.

Repeat a saved run with identical data and seeds in a new output folder. Compare deterministic metrics, grouping and split IDs; runtime timestamps need not match. Then change a seed and ensure the program genuinely reruns rather than loading the old output. Confirm that favourable results are not hard-coded. A code reviewer should be able to regenerate every figure from source results.

Safe error-repair prompt

```
This command failed: [paste command]. Exit status: [number]. Error output:
[paste relevant traceback, with secrets removed]. Diagnose the root cause using
the current files and installed versions. Do not weaken tests, fabricate outputs,
disable TLS, download unverified binaries, or rewrite unrelated modules. Explain
the smallest supported correction, apply it, rerun the affected command and then
the relevant regression checks. State anything still unverified.
```

11. Tests the full project must eventually contain

• Data: target excluded, IDs excluded, licence/provenance saved, split disjointness, preprocessing fitted on train, known schema validated.

• Attribution: known additive game, symmetry/dummy/efficiency, output-scale reconstruction, binary-class selection, explicit failure on unsupported dimensions.

• Grouping: coverage exactly once, complete-linkage condition, constants/missing/categorical cases, higher-rho direction, exact matched group sizes.

• Metrics: RBO identity and hand example, valid range, ties policy, invalid lists, cancellation with zeros and opposing contributions.

• Scientific controls: giant group exposes trivial stability; irrelevant features are detectable; interaction separates coalition sum from exact group value; label randomization changes learned signal.

• Integration: small end-to-end run writes fresh results with correct counts; rerun reproduces deterministic outputs; no stale file is mistaken for the current run.

These are future acceptance requirements. Only the nine checks in the supplied baseline have actually run in this task; do not tell the professor that this complete list is implemented.

12. Presenting the code to Prof. Gupta

Keep the live walkthrough to about two minutes. Show the repository tree and say what each file does. Open `demo.py` at the feature exclusions/split, then the attribution formula and group aggregation. Run `python -m unittest -v test_demo.py`. Open the saved JSON and point to data counts, model family, observed metrics, the matched random comparison and limitations.

Explain one formula rather than scrolling through hundreds of lines. If asked to modify the code, change a configuration such as bootstrap count for a quick run, save to a separate output directory in the final refactor, and explain how it changes compute and evidence. Do not replace ten-fit evidence with a two-fit run while retaining the original figure label.

If asked who wrote the code, say: “I used AI coding assistance, then ran and checked the implementation. I am responsible for understanding and validating it.” Follow university/venue disclosure requirements. Never claim an AI-generated implementation is verified simply because two chat sessions agree.

13. Completion checklist

Tomorrow: four slides reviewed; names edited; pilot and tests run locally or saved output available; key limitations understood; no invented results; exact scope agreed with supervisor.

End of eight weeks: source-linked literature comparison; locked protocol; verified datasets; meaningful negative controls; replicated results; independent replay; clear limitations; advisor-approved manuscript; actual current CFP checked. A polished user interface is optional and should not replace scientific validation.